Immutability & Persistent Collections

ersistent Collections · Immutable playlist edits sharing structure · Build with Waqa

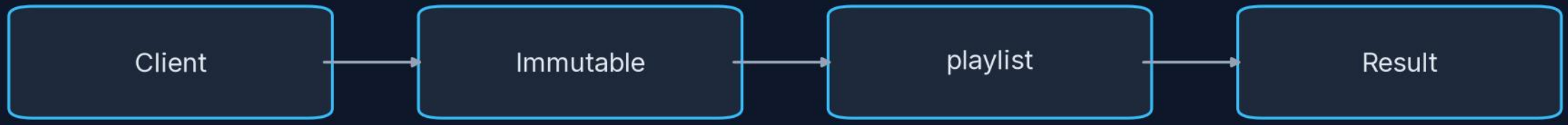
Today's Hook

- Today's force: Immutable playlist edits sharing structure
- We earn Persistent Collections by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Playlist

The Problem

- Scenario: Immutable playlist edits sharing structure
- Constraints: solve «Immutable playlist edits sharing structure» with Persistent Collections — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Immutable playlist edits sharing structure
- Success looks like: [a]

Structure



Persistent Collections: Immutable playlist edits sharing structure

Real-World Scenario

- Running example: Immutable playlist edits sharing structure
- Key types: Playlist
- Happy-path stdout cue: [a]
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.functional.collections.problem;

import java.util.*;

public final class NaivePlaylist {
  public final List<String> songs=new ArrayList<>();
  public void add(String s){ songs.add(s); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Immutable playlist edits sharing structure
- Without Persistent Collections, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.functional.collections.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

final class Playlist {
  private final List<String> songs;
  Playlist(List<String> songs){ this.songs=List.copyOf(songs); }
  Playlist add(String s){ var n=new ArrayList<>(songs); n.add(s); return new Playlist(n); }
  List<String> songs(){ return songs; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.collections.problem;
import java.util.*; import java.util.concurrent.*; import java.util.concurrent.atomic.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Persistent collections: playlist edits ===");
    System.out.println("--- naive ---"); var n=new NaivePlaylist(); n.add("a"); System.out.println(n.songs);
System.out.println("--- persistent ---");
var p=new Playlist(List.of("a")); var p2=p.add("b");
System.out.println(p.songs()); System.out.println(p2.songs());
  }
}
```

Walkthrough Output

```
=== Persistent collections: playlist edits ===
--- naive ---
[a]
--- persistent ---
[a]
[a, b]
```

Step-by-Step Trace

- Match stdout to stages — look for: [a]
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp ... 41-immutability-collections ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Persistent Collections
// Prefer records + sealed where they shrink the demo.
// Problem under test: Immutable playlist edits sharing structure
```

Trade-offs

- Use Persistent Collections when «Immutable playlist edits sharing structure» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Persistent Collections in product language.

Related Patterns

- Name the pattern usually confused with Persistent Collections and separate the forces.
- Sketch the roles for «Immutable playlist edits sharing structure» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Immutable playlist edits sharing structure
- Naive pain → Persistent Collections boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.